

LAPORAN PRAKTIKUM STRUKTUR DATA

“PRATIKUM 9”

Disusun Oleh:

Aryahiyahul Fikra

2511532026

Dosen Pengampu:

Dr. Wahyudi S.T,M.T

Asisten Pembimbing:

Sherly Sukmadira



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

Daftar Isi

Daftar Isi	2
BAB I	3
PENDAHULUAN	3
1.1 Latar Belakang	3
1.2 Tujuan Pratikum	3
BAB II	4
PEMBAHASAN	4
2.1 Binary Tree	4
2.2 Graph	5
2.3 Depth First Search (DFS)	5
2.4 Breadth First Search (BFS)	5
2.5 Analisis Program Binary Tree	6
2.6 Analisis Program Graph Traversal	9
BAB III	11
PENUTUP	11
3.1 Kesimpulan	11
TUGAS PEKAN 9	12
PetaKampus_2511532026	12

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan cara untuk mengorganisasikan data agar dapat diakses dan dikelola secara efisien. Salah satu struktur data non-linear yang banyak digunakan adalah Binary Tree dan Graph. Binary Tree memungkinkan data disusun dalam bentuk hierarki sehingga memudahkan proses pencarian dan penelusuran data. Selain itu, Graph digunakan untuk merepresentasikan hubungan antar simpul yang lebih kompleks.

Pada praktikum ini dilakukan implementasi Binary Tree menggunakan bahasa pemrograman Java dengan operasi pembuatan simpul, perhitungan jumlah simpul, serta traversal pohon menggunakan metode Inorder, Preorder, dan Postorder. Selain itu, dilakukan implementasi Graph Traversal menggunakan algoritma Depth First Search (DFS) dan Breadth First Search (BFS) untuk menelusuri simpul-simpul pada graf.

Melalui praktikum ini diharapkan mahasiswa dapat memahami konsep struktur data non-linear serta mengimplementasikannya dalam bentuk program Java.

1.2 Tujuan Pratikum

Tujuan pratikum ini Adalah:

1. Memahami konsep Binary Tree.
2. Memahami konsep Graph.
3. Mengimplementasikan Binary Tree dalam bahasa Java.
4. Mengimplementasikan traversal Binary Tree menggunakan metode Inorder, Preorder, dan Postorder.
5. Mengimplementasikan algoritma DFS dan BFS pada Graph.
6. Menganalisis hasil traversal yang diperoleh dari program.

BAB II

PEMBAHASAN

2.1 Binary Tree

Binary Tree adalah struktur data berbentuk pohon dimana setiap simpul memiliki maksimal dua anak, yaitu left child dan right child. Simpul pertama disebut root.

Operasi yang umum dilakukan pada Binary Tree antara lain:

- Menambahkan simpul.
- Menghitung jumlah simpul.
- Mencari data.
- Traversal pohon.

Traversal Binary Tree

1. Inorder

Traversal dilakukan dengan urutan:

Kiri → Root → Kanan

Contoh:

4 2 5 1 6 3 7

2. Preorder

Traversal dilakukan dengan urutan:

Root → Kiri → Kanan

Contoh:

1 2 4 5 3 6 7

3. Postorder

Traversal dilakukan dengan urutan:

Kiri → Kanan → Root

Contoh:

4 5 2 6 7 3 1

2.2 Graph

Graph adalah struktur data yang terdiri dari kumpulan simpul (vertex) dan sisi (edge) yang menghubungkan antar simpul.

Representasi graph yang digunakan pada program ini adalah Adjacency List menggunakan struktur data HashMap.

2.3 Depth First Search (DFS)

DFS merupakan algoritma penelusuran graf yang bekerja dengan menelusuri satu cabang hingga kedalaman maksimum sebelum kembali ke simpul sebelumnya.

Karakteristik DFS:

1. Menggunakan konsep rekursi atau stack.
2. Menjelajahi simpul sedalam mungkin terlebih dahulu.
3. Cocok untuk pencarian jalur dan deteksi siklus.

2.4 Breadth First Search (BFS)

BFS merupakan algoritma penelusuran graf yang mengunjungi seluruh tetangga pada suatu level sebelum berpindah ke level berikutnya.

Karakteristik BFS:

1. Menggunakan Queue.
2. Menelusuri graf per tingkat (level).
3. Cocok untuk mencari lintasan terpendek pada graf tak berbobot.

2.5 Analisis Program Binary Tree

Program terdiri dari tiga class utama:

1. Node_2511532026

```
3 public class Node_2511532026 {
4     int data_2026;
5     Node_2511532026 left_2026;
6     Node_2511532026 right_2026;
7     public Node_2511532026(int data_2026) {
8         this.data_2026 = data_2026;
9         left_2026 = null;
10        right_2026 = null;
11    }
12
13    public void setLeft_2026(Node_2511532026 node_2026) {
14        if (left_2026 == null)
15            left_2026 = node_2026;
16    }
17    public void setRight_2026(Node_2511532026 node_2026) {
18        if (right_2026 == null)
19            right_2026 = node_2026;
20    }
21    public Node_2511532026 getLeft_2026() {
22        return left_2026;
23    }
24    public Node_2511532026 getRight_2026() {
25        return right_2026;
26    }
27    public int getData_2026() {
28        return data_2026;
29    }
30
31    public void setData_2026 (int data_2026) {
32        this.data_2026 = data_2026;
33    }
34    void printPreorder_2026(Node_2511532026 node_2026) {
35        if (node_2026==null)
36            return;
37        System.out.print(node_2026.data_2026 + " ");
38        printPreorder_2026(node_2026.left_2026);
39        printPreorder_2026(node_2026.right_2026);
40    }
41    void printPostorder_2026(Node_2511532026 node_2026) {
42        if (node_2026 == null)
43            return;
44        printPostorder_2026(node_2026.left_2026);
45        System.out.print(node_2026.data_2026 + " ");
46        printPostorder_2026(node_2026.right_2026);
47    }
48
49    void printInorder_2026(Node_2511532026 node_2026) {
50        if (node_2026 == null)
51            return;
52        printInorder_2026(node_2026.left_2026);
53        System.out.print(node_2026.data_2026 + " ");
54        printInorder_2026(node_2026.right_2026);
55    }
56    public String print() {
57        return this.print("",true,"");
58    }
59
60    public String print(String prefix,boolean isTail,String sb) {
61        if(right_2026 != null) {
62            right_2026.print(prefix+ (isTail ? "| " : " "),false,sb);
63        }
64        System.out.println(prefix + (isTail ? " \\-- " : "/--")+data_2026);
65        if(left_2026!=null) {
66            left_2026.print(prefix+(isTail ? " " : "| "),true,sb);
67        }
68        return sb;
69    }
```

Class ini digunakan untuk merepresentasikan simpul pada Binary Tree.

Atribut:

int data_2026;

Node_2511532026 left_2026;

Node_2511532026 right_2026;

Fungsi utama:

- Menyimpan data simpul.
- Menyimpan referensi anak kiri dan kanan.
- Menampilkan traversal Inorder, Preorder, dan Postorder.

2. BTree_2511532026

```
1 package pekan9_2511532026;
2
3 public class BTree_2511532026 {
4     private Node_2511532026 root_2026;
5     private Node_2511532026 currentNode_2026;
6
7     public BTree_2511532026() {
8         root_2026 = null;
9     }
10
11    public boolean search_2026(int data_2026) {
12        return search_2026(root_2026, data_2026);
13    }
14
15    private boolean search_2026(Node_2511532026 node_2026, int data_2026) {
16        if (node_2026.getData_2026() == data_2026)
17            return true;
18
19        if (node_2026.getLeft_2026() != null)
20            if (search_2026(node_2026.getLeft_2026(), data_2026))
21                return true;
22
23        if (node_2026.getRight_2026() != null)
24            if (search_2026(node_2026.getRight_2026(), data_2026))
25                return true;
26
27        return false;
28    }
29
30    public void printInorder_2026() {
31        root_2026.printInorder_2026(root_2026);
32    }
33
34    public void printPreOrder_2026() {
35        root_2026.printPreorder_2026(root_2026);
36    }
37
38    public void printPostOrder_2026() {
39        root_2026.printPostorder_2026(root_2026);
40    }
41
42    public Node_2511532026 getRoot_2026() {
43        return root_2026;
44    }
45    public boolean isEmpty_2026() {
46        return root_2026 == null;
47    }
48
49    public int countNodes_2026() {
50        return countNodes_2026(root_2026);
51    }
52
53    private int countNodes_2026(Node_2511532026 node_2026) {
54        int count_2026 = 1;
55
56        if (node_2026 == null) {
57            return 0;
58        } else {
59            count_2026 += countNodes_2026(node_2026.getLeft_2026());
60            count_2026 += countNodes_2026(node_2026.getRight_2026());
61            return count_2026;
62        }
63    }
64
65    public void print_2026() {
66        root_2026.print();
67    }
68
69    public Node_2511532026 getCurrent_2026() {
70        return currentNode_2026;
71    }
72
73    public void setCurrent_2026(Node_2511532026 node_2026) {
74        this.currentNode_2026 = node_2026;
75    }
76
77    public void setRoot_2026(Node_2511532026 root_2026) {
78        this.root_2026 = root_2026;
79    }
80}
```

Class ini digunakan untuk mengelola Binary Tree.

Method penting:

Method

search_2026()

countNodes_2026()

printInorder_2026()

printPreOrder_2026()

printPostOrder_2026()

Fungsi

Mencari data pada pohon

Menghitung jumlah simpul

Menampilkan traversal Inorder

Menampilkan traversal Preorder

Menampilkan traversal Postorder

3. BTreeDriver_2511532026

```
1 package pekan9_2511532026;
2
3 public class BTreeDriver_2511532026 {
4
5     public static void main(String[] args) {
6
7         BTree_2511532026 tree_2026 = new BTree_2511532026();
8
9         System.out.print("Jumlah Simpul awal pohon: ");
10        System.out.println(tree_2026.countNodes_2026());
11
12        Node_2511532026 root_2026 = new Node_2511532026(1);
13        tree_2026.setRoot_2026(root_2026);
14
15        System.out.print("Jumlah simpul jika hanya ada root: ");
16        System.out.println(tree_2026.countNodes_2026());
17
18        Node_2511532026 node2_2026 = new Node_2511532026(2);
19        Node_2511532026 node3_2026 = new Node_2511532026(3);
20        Node_2511532026 node4_2026 = new Node_2511532026(4);
21        Node_2511532026 node5_2026 = new Node_2511532026(5);
22        Node_2511532026 node6_2026 = new Node_2511532026(6);
23        Node_2511532026 node7_2026 = new Node_2511532026(7);
24        Node_2511532026 node8_2026 = new Node_2511532026(8);
25        Node_2511532026 node9_2026 = new Node_2511532026(9);
26
27        root_2026.setLeft_2026(node2_2026);
28        node2_2026.setLeft_2026(node4_2026);
29        node2_2026.setRight_2026(node5_2026);
30        node4_2026.setRight_2026(node8_2026);
31        root_2026.setRight_2026(node3_2026);
32        node3_2026.setLeft_2026(node6_2026);
33        node3_2026.setRight_2026(node7_2026);
34        node6_2026.setLeft_2026(node9_2026);
35        tree_2026.setCurrent_2026(tree_2026.getRoot_2026());
36        System.out.println("menampilkan simpul terakhir: ");
37        System.out.println(tree_2026.getCurrent_2026().getData_2026());
38
39        System.out.println("Jumlah simpul setelah simpul 7 ditambahkan");
40        System.out.println(tree_2026.countNodes_2026());
41
42        System.out.println("InOrder: ");
43        tree_2026.printInorder_2026();
44
45        System.out.println("\nPreorder: ");
```

Class driver yang digunakan untuk membuat objek Binary Tree dan menghubungkan seluruh simpul.

Hasil Traversal

Inorder

4 8 2 5 1 9 6 3 7

Preorder

1 2 4 8 5 3 6 9 7

Postorder

8 4 5 2 9 6 7 3 1

Jumlah simpul:

9 simpul

Output:

```
<terminated> BTreeDriver_2511532026 [Java Application]
Jumlah simpul setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
Preorder:
1 2 4 8 5 3 6 9 7
Postorder:
8 4 5 2 9 6 7 3 1
menampilkan simpul dalam bentuk pohon
|
|--7
|
|--3
|   |--6
|   |--9
|--1
|   |--5
|   |--2
|       |--8
|       |--4
```


2.6 Analisis Program Graph Traversal

Program menggunakan struktur:

Map<String, List<String>>

untuk menyimpan adjacency list.

Adjacency List

A -> B, C

B -> A, D, E

C -> A

D -> B

E -> B

Traversal DFS

Urutan kunjungan:

A B D E C

Penjelasan:

1. Mulai dari A.
2. Ke B.
3. Ke D.
4. Kembali ke B lalu ke E.
5. Kembali ke A lalu ke C.

Traversal BFS

Urutan kunjungan:

A B C D E

Penjelasan:

1. Mulai dari A.
2. Kunjungi semua tetangga A yaitu B dan C.
3. Kunjungi tetangga B yaitu D dan E.

Kode Program:

```
1 package pekan9_2511532026;
2 import java.util.*;
3
4 public class GraphTraversal_2511532026 {
5
6     private Map<String, List<String>> graph_2026 = new HashMap<>();
7
8     // Menambahkan edge (graf tak berarah)
9     public void addEdge_2026(String node1_2026, String node2_2026) {
10         graph_2026.putIfAbsent(node1_2026, new ArrayList<>());
11         graph_2026.putIfAbsent(node2_2026, new ArrayList<>());
12
13         graph_2026.get(node1_2026).add(node2_2026);
14         graph_2026.get(node2_2026).add(node1_2026);
15     }
16
17     // Menampilkan graf awal
18     public void printGraph_2026() {
19         System.out.println("Graf Awal (Adjacency List):");
20
21         for (String node_2026 : graph_2026.keySet()) {
22             System.out.print(node_2026 + " -> ");
23
24             List<String> neighbors_2026 = graph_2026.get(node_2026);
25             System.out.println(String.join(", ", neighbors_2026));
26         }
27
28         System.out.println();
29     }
30
31     // DFS rekursif
32     public void dfs_2026(String start_2026) {
33         Set<String> visited_2026 = new HashSet<>();
34
35         System.out.println("Penelusuran DFS:");
36         dfsHelper_2026(start_2026, visited_2026);
37
38         System.out.println();
39     }
40
41     private void dfsHelper_2026(String current_2026, Set<String> visited_2026) {
42         if (visited_2026.contains(current_2026)) return;
43
44         visited_2026.add(current_2026);
45
46         for (String neighbor_2026 : graph_2026.getOrDefault(current_2026, new ArrayList<>())) {
47             dfsHelper_2026(neighbor_2026, visited_2026);
48         }
49     }
50
51     public void bfs_2026(String start_2026) {
52         Set<String> visited_2026 = new HashSet<>();
53         Queue<String> queue_2026 = new LinkedList<>();
54
55         queue_2026.add(start_2026);
56         visited_2026.add(start_2026);
57
58         System.out.println("Penelusuran BFS:");
59
60         while (!queue_2026.isEmpty()) {
61             String current_2026 = queue_2026.poll();
62             System.out.print(current_2026 + " ");
63
64             for (String neighbor_2026 : graph_2026.getOrDefault(current_2026, new ArrayList<>())) {
65                 if (!visited_2026.contains(neighbor_2026)) {
66                     queue_2026.add(neighbor_2026);
67                     visited_2026.add(neighbor_2026);
68                 }
69             }
70
71             System.out.println();
72         }
73
74         System.out.println();
75     }
76
77     public static void main(String[] args) {
78         GraphTraversal_2511532026 graph_2026 = new GraphTraversal_2511532026();
79
80         // Contoh graf: A-B, A-C, B-D, B-E
81         graph_2026.addEdge_2026("A", "B");
82         graph_2026.addEdge_2026("A", "C");
83         graph_2026.addEdge_2026("B", "D");
84         graph_2026.addEdge_2026("B", "E");
85
86         // Cetak graf awal
87         System.out.println("Graf Awal adalah: ");
88         graph_2026.printGraph_2026();
89
90         // DFS
91         graph_2026.dfs_2026("A");
92     }
93 }
```

<terminated> GraphTraversal_2511532026 [Ja

Graf Awal adalah:
Graf Awal (Adjacency List):

A -> B, C
B -> A, D, E
C -> A
D -> B
E -> B

Penelusuran DFS:

A B D E C

Penelusuran BFS:

A B C D E

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. Binary Tree merupakan struktur data non-linear yang menyimpan data dalam bentuk hierarki.
2. Traversal Inorder, Preorder, dan Postorder menghasilkan urutan kunjungan simpul yang berbeda.
3. DFS menelusuri graf hingga kedalaman maksimum terlebih dahulu menggunakan rekursi.
4. BFS menelusuri graf berdasarkan level menggunakan Queue.
5. Implementasi Binary Tree dan Graph Traversal dalam Java berhasil dijalankan dengan baik dan menghasilkan output sesuai teori.

TUGAS PEKAN 9

PetaKampus_2511532026

Deskripsi Peta Kampus

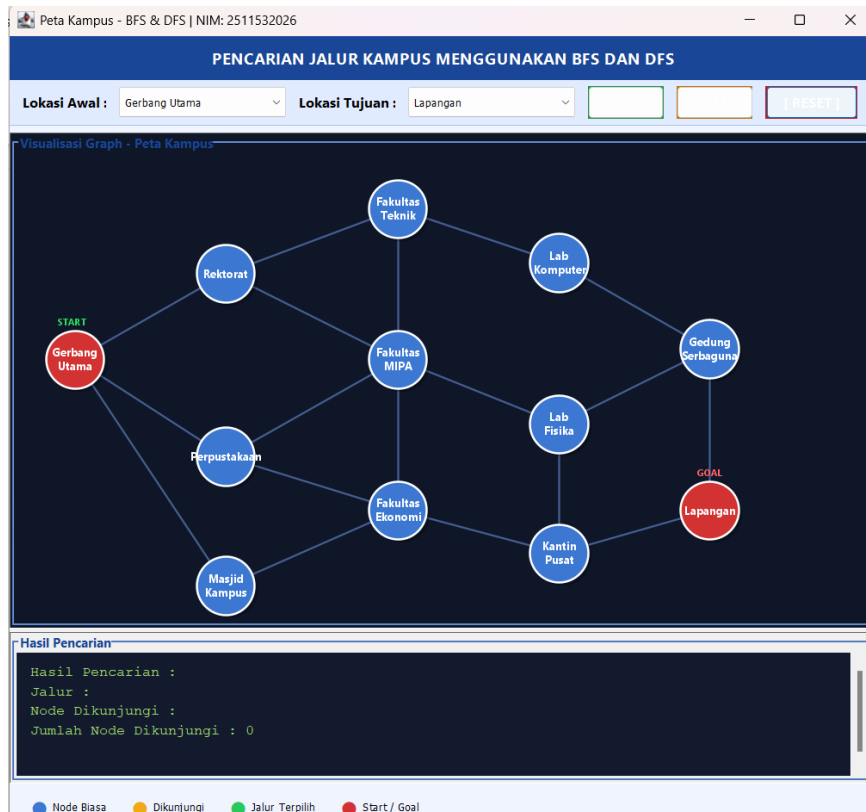
Peta yang digunakan dalam program ini merepresentasikan lingkungan sebuah kampus dengan 12 lokasi (node) dan 18 jalur penghubung (edge). Setiap node merepresentasikan lokasi nyata yang umum ditemukan di dalam area kampus, dan setiap edge merepresentasikan jalur/jalan yang menghubungkan dua lokasi tersebut secara langsung.

Struktur Graph

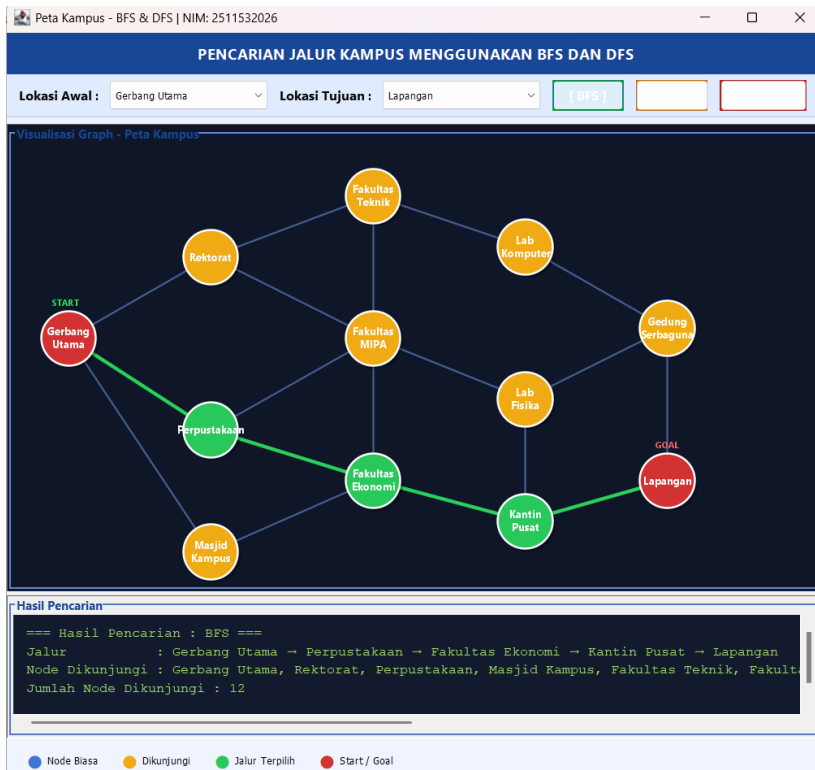
Graph pada program ini bersifat **tidak berarah (undirected graph)**, artinya setiap jalur yang menghubungkan dua lokasi dapat dilalui dari dua arah. Misalnya, jika terdapat jalur dari Gerbang Utama ke Rektorat, maka jalur dari Rektorat ke Gerbang Utama juga tersedia.

Graph ini juga bersifat **tidak berbobot (unweighted graph)**, sehingga setiap jalur dianggap memiliki jarak/biaya yang sama. Oleh karena itu, BFS mampu menemukan jalur dengan jumlah simpul paling sedikit (jalur terpendek berdasarkan hop count).

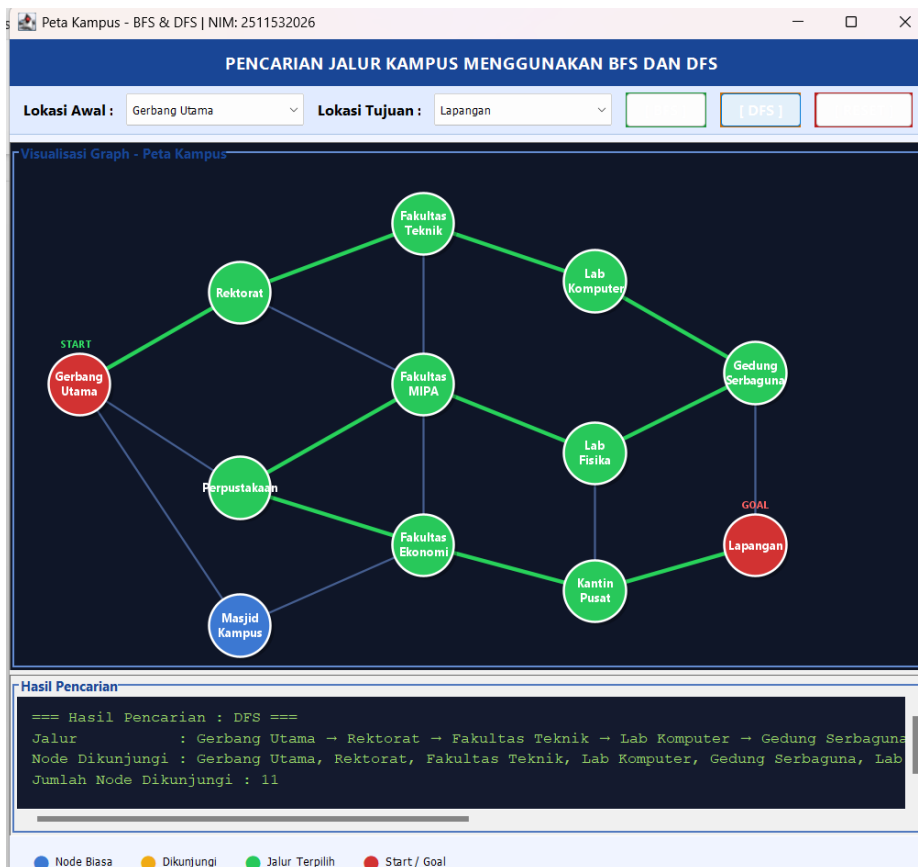
Output:



Hasil BFS:



Hasil DFS:



Breadth First Search (BFS)

BFS melakukan pencarian dengan menjelajahi semua node tetangga pada level/kedalaman yang sama terlebih dahulu sebelum berpindah ke level berikutnya. BFS menggunakan struktur data **Queue (antrian)** dengan prinsip FIFO (First In First Out).

Karakteristik BFS pada Peta Kampus ini:

- Menjamin ditemukannya **jalur terpendek** (shortest path) berdasarkan jumlah edge/simpul yang dilalui
- Mengeksplorasi node secara **melebar** dari titik awal
- Membutuhkan **memori lebih besar** karena menyimpan semua node pada level yang sama di dalam queue
- Cocok digunakan ketika tujuan berada **dekat dengan titik awal**

Depth First Search (DFS)

DFS melakukan pencarian dengan menjelajahi satu jalur sedalam mungkin hingga menemukan dead end, lalu backtrack dan mencoba jalur lain. DFS menggunakan struktur data **Stack (tumpukan)** dengan prinsip LIFO (Last In First Out).

Karakteristik DFS pada Peta Kampus ini:

- **Tidak menjamin** jalur terpendek yang ditemukan
- Mengeksplorasi node secara **mendalam** mengikuti satu cabang hingga habis
- Membutuhkan **memori lebih kecil** dibanding BFS karena hanya menyimpan node pada jalur yang sedang dieksplorasi
- Cocok digunakan ketika tujuan berada **jauh dan dalam** dari titik awal